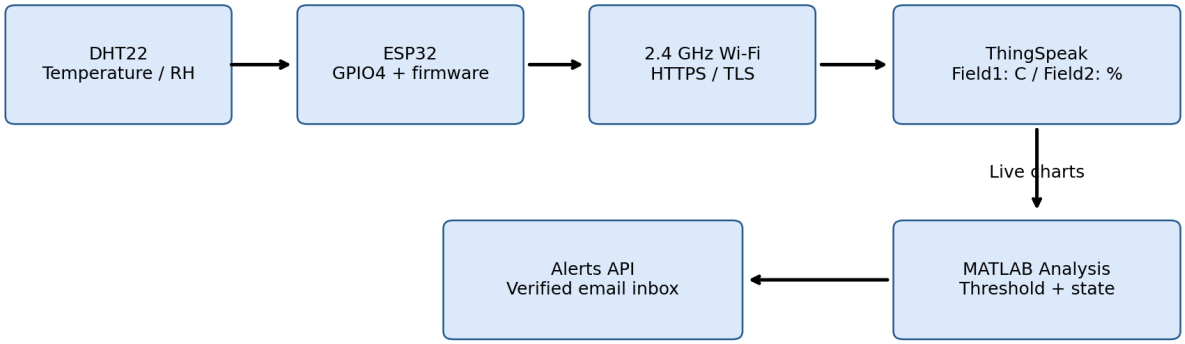
IoT-Based Weather Station

Using ESP32, DHT22 and ThingSpeak Cloud

Mini Project Report and Implementation Package

IoT Weather Station — system architecture



Architecture diagram; not evidence of a connected cloud system

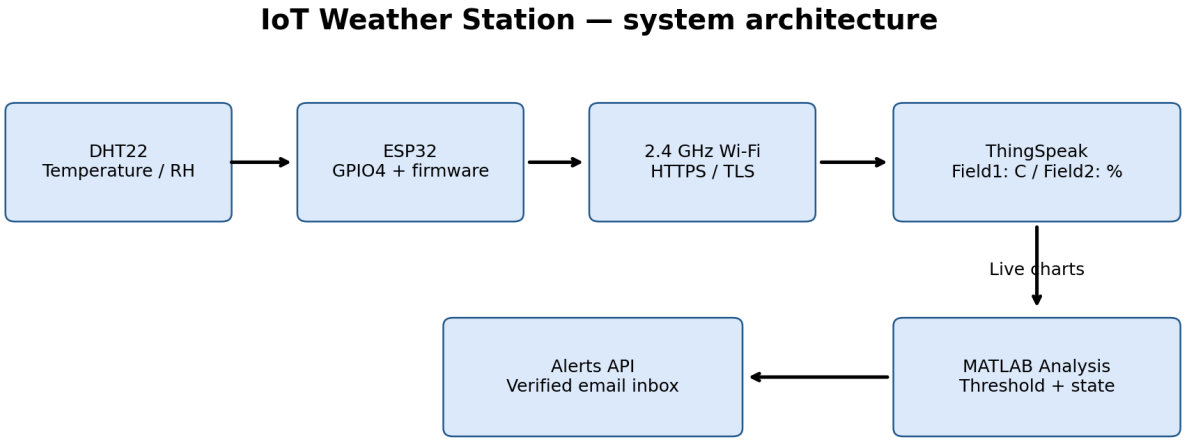
Student: Deep Bisen

Date: 17 October 2026

Submission status: firmware, cloud script, wiring design, dashboard setup and software-only validation are supplied. Physical hardware testing, ESP32 target compilation, actual ThingSpeak updates and delivered-email evidence remain pending. This report does not claim those operations were completed.

Abstract: The proposed station measures ambient temperature and relative humidity with a DHT22, validates samples on an ESP32, uploads them securely over Wi-Fi, and visualizes them in ThingSpeak. A scheduled MATLAB Analysis checks high limits and sends an email through the ThingSpeak Alerts API. Hysteresis, persistent state and cooldown reduce repeated alerts. No Wi-Fi password or cloud key is included in this submission.

1. System architecture (Q1)



Architecture diagram; not evidence of a connected cloud system

The DHT22 supplies digital temperature and humidity readings to GPIO4. ESP32 firmware samples every 2.5 seconds and rejects invalid readings. Every 20 seconds it attempts an authenticated HTTPS write to ThingSpeak using field1 for temperature and field2 for humidity. Wi-Fi provides Internet access; a trusted CA and correct clock are prerequisites for TLS verification.

ThingSpeak stores timestamped entries and displays two field charts. A TimeControl job periodically runs MATLAB Analysis. It reads the newest weather entry, rejects stale data, applies thresholds, and invokes the Alerts API. A separate private state channel stores the alert latch and last-send timestamp. Email is delivered by the cloud service to the verified account inbox; the ESP32 does not need SMTP credentials.

Element	Function / design choice
DHT22	Temperature and RH; sample interval 2.5 s
ESP32	Validation, Wi-Fi reconnect, secure HTTP upload
Weather channel	Field1: temperature (C); Field2: humidity (%)
State channel	Field1: latch; Field2: last alert Unix time
MATLAB + TimeControl	Scheduled thresholds, hysteresis and cooldown
ThingSpeak Alerts API	Email request to the account email destination

Data path: sensing -> validation -> upload -> entry acknowledgment -> chart -> scheduled threshold decision -> email. A positive API entry ID proves acceptance of a write, while a visible received message is required to prove email delivery. Neither is demonstrated by a local diagram.

2. Sensor interfacing and firmware (Q2)

Wiring reference — verify module pin labels before connecting

DHT22 bare sensor pin	ESP32 connection	Purpose
1 — VCC	3V3	Supply
2 — DATA	GPIO4	Digital readings
3 — NC	No connection	Unused
4 — GND	GND	Common ground
External resistor	DATA to 3V3: 4.7–10 kOhm	Pull-up
100 nF capacitor	VCC to GND near sensor	Decoupling

Use a 3.3 V supply and common ground. A bare four-pin DHT22 uses the pin order shown when viewed from the grille side; breakout modules can differ. Place a 4.7–10 kOhm DATA pull-up to 3V3 and a 100 nF decoupling capacitor at the sensor. Power down before wiring. The sensor and all GPIO connections must stay within the chosen board/module ratings.

Firmware initializes Serial at 115200 baud, DHT22, station-mode Wi-Fi and NTP time synchronization. Every 2500 ms it reads humidity and temperature. validReading rejects NaN, infinity, temperature outside -40 to 80 C, or humidity outside 0–100%. A failed reading invalidates the current sample rather than uploading a cached value as if it were fresh.

The upload loop uses unsigned elapsed-time arithmetic, which continues working across millis() wraparound. Uploads are attempted at least 20 seconds apart, including failures. The firmware requests reconnection every 15 seconds while offline. It skips uploads when disconnected, when the current sample is invalid, when the clock is not initialized, or when the trusted root CA is absent. Failed uploads are not queued; this simple live-monitoring design can have data gaps during outages.

Files: WeatherStation.ino contains full firmware; logic.h contains testable validation/timing helpers; config.example.h is copied locally to config.h for secrets and the current server trust root. platformio.ini specifies esp32dev and library versions. Arduino IDE is an alternative. ESP32/DHT22 library compilation and hardware upload are not verified in this environment.

3. Secure cloud upload and field mapping (Q3)

Create a private weather channel and enable Field 1: Temperature (C), Field 2: Relative Humidity (%). Copy the channel Write API key only into local config.h. Firmware POSTs URL-encoded form data to https://api.thingspeak.com/update. Example payload structure: api_key=REDACTED&field1=28.50&field2=62.00. The example values are illustrative, not measured.

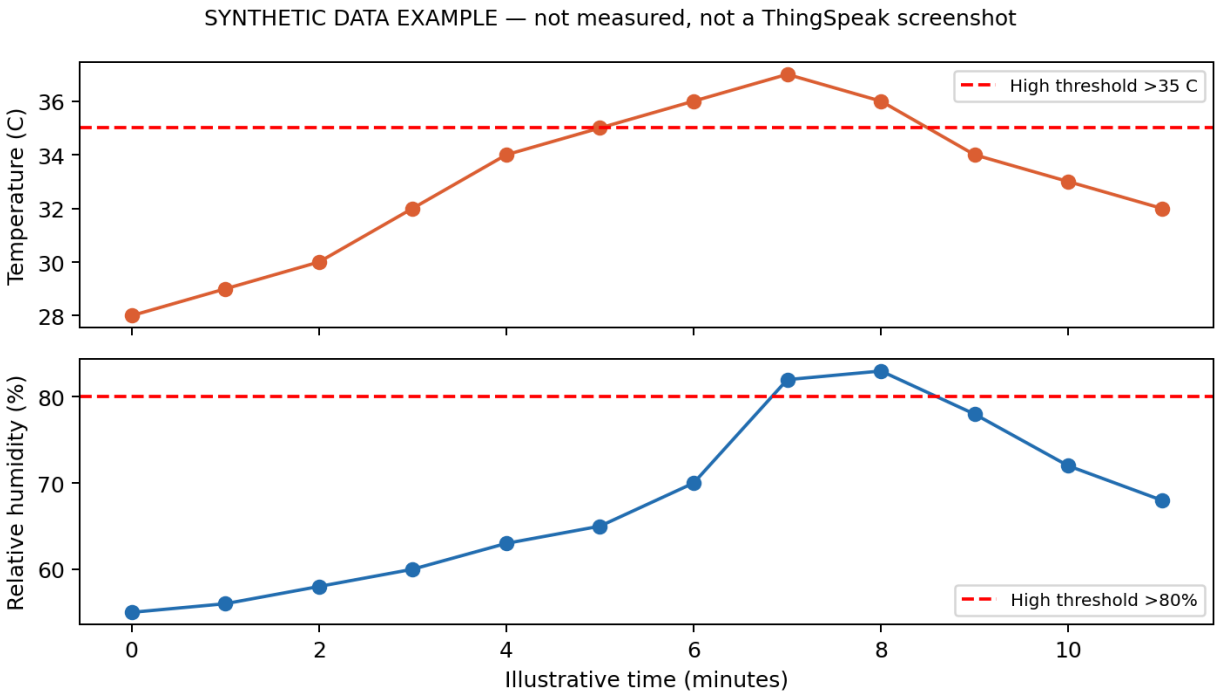
Use a trusted root PEM for the current api.thingspeak.com certificate chain, obtained from the issuing CA over verified HTTPS. The supplied firmware deliberately refuses upload if the root is missing and never disables TLS verification. Set network time before the handshake. Root rotation may require updating local trust. A complete current certificate is not embedded because the live chain could not be inspected here.

Acceptance criteria: response HTTP 200 AND a positive numeric entry ID. Body 0 means rejection even with HTTP 200. Match the acknowledged entry with the cloud timestamp and field values using the private channel view or its authenticated read API. Do not print or screenshot API keys. Use a 20-second write period and confirm the current account quota; the commonly documented free-channel minimum is 15 seconds.

Test / condition	Expected result	Actual status
Valid sample + Wi-Fi + TLS	Positive entry ID; matching fields	Pending hardware/cloud
Invalid sample	No upload of invalid data	Helper logic tested
Wrong Write API key	Rejected write, logged failure	Not executed
Missing root CA / unset clock	Upload refused until configured	Source reviewed; target unrun
Wi-Fi interruption	Retry; resume when connected	Not executed

In this machine a read-only HTTPS request to api.thingspeak.com failed with proxy CONNECT HTTP 403. That is a network-access blocker, not proof that a particular ThingSpeak account or credential is invalid. No private account keys were supplied. Therefore cloud field updates could not be verified here.

4. Data visualization (Q4)



The above figure is a synthetic visualization example, not a live ThingSpeak screenshot. Its source values are included in verification/synthetic_example.csv and must not be represented as measured results.

Configure Private View with a Field 1 temperature line chart, Y-axis Temperature (C), X-axis time, and a Field 2 humidity chart, Y-axis Relative Humidity (%). Choose a suitable recent time window and enable supported automatic refresh. Keep the channel private unless publication is explicitly intended. Add a channel description identifying GPIO4/DHT22 and the 20-second upload interval.

Verification procedure: record a normal baseline, gently change the local environment without exceeding sensor limits, and confirm both serial values and cloud entries change consistently. Dashboard refresh and Internet latency mean the graphs are near-real-time. No sampling-accuracy claim can be made without a reference instrument and measured comparisons.

Required screenshot: real channel dashboard showing both labelled graphs, recent timestamps and several entries. Redact API keys and unnecessary personal information. Place it in evidence/03_dashboard.png and replace this illustrative discussion with measured observations after testing.

5. Email alert implementation (Q5)

Default trigger: temperature >35 C OR humidity >80%. These are demonstration thresholds, not certified safety limits. The MATLAB script reads the newest sample and ignores data more than 120 seconds old. It sends an Alerts API email only when the system is unlatched and the last-send cooldown of 1800 seconds has elapsed.

After an accepted email request, the state channel records latch=1 and lastSent=Unix time. The script rearms only when BOTH temperature <=34 C AND humidity <=78%. Persistent high conditions therefore do not send an email every minute. Equality at the high threshold does not trigger. A high event after recovery but within the cooldown waits for the cooldown before sending if the event persists.

Configure the private script cloud/threshold_email.m with channel IDs, weather Read key, state Read/Write keys and Alerts API key. Initialize state channel Field1=0, Field2=0 before scheduling. Execute once manually, then configure TimeControl to run every minute if the account supports it. The recipient is the verified account email used by the Alerts service. Check current account API availability and delivery quotas.

The endpoint is https://api.thingspeak.com/alerts/send with the ThingSpeak-Alerts-API-Key header. The script sends subject/body only when conditions require it. A successful API call is an accepted request, not proof of inbox delivery. A failure after sending but before saving state can produce a duplicate later because the two operations are not atomic. Unavailable state data stops processing rather than guessing a cooldown.

Scenario	Expected action
25 C, 50% RH	No alert
35 C, 80% RH	No alert: strict high-limit comparison
36 C, 50% RH, unlatched, cooldown over	Send one request and latch
Still high and already latched	No repeat request
34 C, 78% RH, latched	Rearm; preserve last-send time
New high within 30 min	Wait until cooldown expires
Stale / invalid sample	Ignore, no alert decision

6. Verification, results and limitations

Two software-only checks were executed: the actual C++ validation/timing helpers passed 14 assertions, and a Python reference model of the alert policy passed 12 cases. Tests cover sensor range boundaries, non-finite data, upload interval edges, millis rollover, high limits, recovery hysteresis, cooldown and stale-data rejection. The Python model tests policy logic; it does not execute MATLAB or verify the cloud API.

Verification item	Outcome
C++ helpers: g++ C++11, warnings enabled	PASS — 14 assertions
Python alert-policy reference	PASS — 12 cases
ESP32 firmware target compile / flash	NOT RUN — toolchain/hardware unavailable
DHT22 physical readings	NOT RUN — no attached sensor
ThingSpeak write/read and live charts	BLOCKED — proxy 403; keys unavailable
MATLAB execution and TimeControl	NOT RUN — account setup required
Actual delivered email	NOT RUN — real evidence required

Expected example, not observed output: a valid 36 C / 50% reading should update fields 1/2 and, after scheduled evaluation and subject to state/cooldown, request one email. No numeric measurement series, live dashboard capture, or received email is claimed in this report.

Security and reliability: local secrets are excluded by .gitignore; share only the config template. Validate TLS and server identity, avoid key-bearing URLs in screenshots, and keep state private. Sensor wiring faults, invalid readings, Wi-Fi drops and rejected writes are handled or logged. Pending improvements include outage buffering, independent calibration, watchdog/liveness alerts and more complete fault logging.

To finish validation: wire the device, fill local settings and trusted root, compile and upload, confirm several positive entry IDs, capture corresponding charts, provoke a clearly labelled controlled high-limit event, capture its email, verify suppression and rearming, then restore default thresholds. Add actual evidence using evidence/README.md. These actions require real hardware/account access; software-only tests cannot substitute for them.

7. Conclusion and submission checklist

A complete software implementation and documented architecture have been prepared for the ESP32/DHT22 ThingSpeak weather station. The design covers sensing, validated secure upload, live-chart configuration and an email alert policy with hysteresis and cooldown. Tested local helpers behave as expected. End-to-end functionality remains unverified until the device and configured ThingSpeak account are run; the required real screenshots remain outstanding.

Assignment requirement	Included deliverable / remaining action
Q1 architecture	Block diagram and explanation
Q2 sensor interface	Wiring reference and full .ino firmware
Q3 cloud upload	HTTPS firmware and acceptance procedure; live check pending
Q4 visualization	Chart setup + labelled synthetic example; real screenshot pending
Q5 email	MATLAB script and configuration; actual delivery pending
Report PDF	This report, plus editable Word version
Source code	firmware/ and cloud/ folders
Dashboard and alert screenshots	Not available; add real evidence before final submission

Folder: IoT_Weather_Station_Deep_Bisen. Add real screenshots, update results with actual observations, then ZIP this folder and upload it to Google Drive. Share the instructor-accessible Drive link as required by the assignment. A prepared package without the pending evidence does not meet every submission requirement yet.

References:

1. Supplied IoT Mini Project statement (ESP32/DHT22/ThingSpeak requirements).
2. MathWorks ThingSpeak documentation: https://www.mathworks.com/help/thingspeak/
3. ThingSpeak Alerts API: https://www.mathworks.com/help/thingspeak/alerts-api.html
4. Espressif Arduino-ESP32 documentation:
https://docs.espressif.com/projects/arduino-esp32/en/latest/
5. Adafruit DHT sensor library: https://github.com/adafruit/DHT-sensor-library

Consult current official documentation for certificate chains, UI names, API quotas and supported scheduling intervals.